

# Ingenierie Vision-Langage

CLIP, Qwen3-VL, comprehension documentaire et RAG multimodal

**GHANMI Helmi**

AI Engineer | Former Researcher in Applied Mathematics

<https://www.researchgate.net/profile/Ghanmi-Helmi>

Runtime de reference : Python 3.12

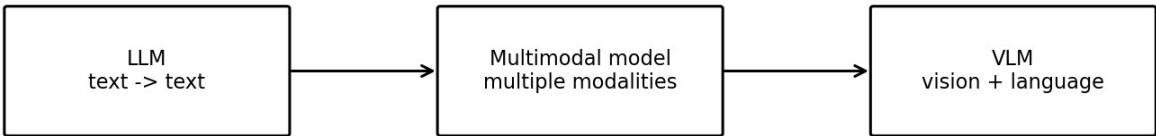
Notes modeles/API verifiees le 27 aout 2026

# Comment utiliser ce cours

Ce cours part de la difference entre modeles de langage et modeles vision-langage, puis relie l'utilisation des modeles a une architecture de projet orientee production. L'objectif n'est pas seulement d'executer un modele, mais de comprendre comment la preuve visuelle entre dans le retrieval et la generation fondee sur des sources.

- Comprendre LLM, modele multimodal, VLM, OCR, embeddings et RAG.
- Utiliser CLIP pour la similarite image-texte et la classification zero-shot.
- Executer Qwen3-VL-2B-Instruct depuis le Hub/cache Hugging Face ou depuis un snapshot local hors-ligne.
- Transformer diagrammes, captures et pages de documents en chunks structures et tracables.
- Commencer par un retrieval texte puis evoluer vers embedding et reranking multimodaux Qwen3-VL.
- Evaluer extraction, retrieval et reponse finale avant le passage a grande echelle.

## 1. LLM, modele multimodal et VLM



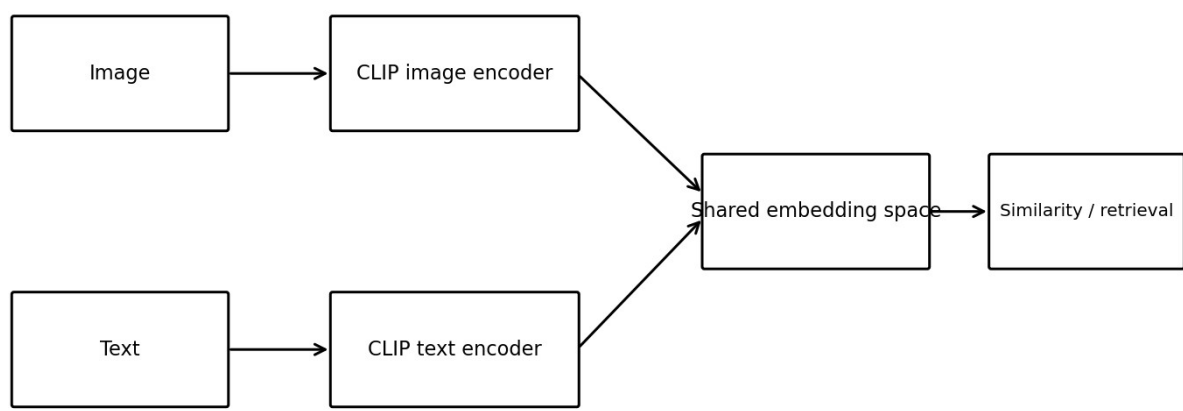
VLM is a multimodal specialization focused on visual-language understanding.

OCR recognizes characters. A VLM can also reason about layout, arrows, groups, tables and visual relationships.

| Terme             | Definition simple                              | Role typique                                              |
|-------------------|------------------------------------------------|-----------------------------------------------------------|
| LLM               | Principalement tokens texte -> tokens texte    | Raisonnement/generation une fois les preuves textualisees |
| Modele multimodal | Famille generale acceptant plusieurs modalites | Combinaisons texte, image, audio, video                   |
| VLM               | Specialisation vision + langage                | Pages, figures, captures, diagrammes, VQA                 |
| OCR               | Reconnaissance de caracteres                   | Recuperer le texte litteral, pas forcément les relations  |
| Embedding         | Projection vers des vecteurs                   | Similarite et retrieval                                   |
| RAG               | Retrieval avant generation                     | Fonder la reponse sur les bons chunks                     |

Idee cle : l'OCR peut lire les labels d'un schema, alors qu'un VLM peut aussi expliquer qu'une fleche relie Service A a PostgreSQL. Pour le document AI, cette relation visuelle est souvent l'information decisive.

## 2. CLIP : la base de l'espace d'embeddings partage



Qwen3-VL instead consumes image + instruction and generates an answer, description or structured output.

CLIP apprend des representations image et texte comparables dans un meme espace. Il est tres utile pour similarite, retrieval et classification zero-shot. Ce n'est pas un modele generatif conversationnel : CLIP calcule des scores, Qwen3-VL genere du langage conditionne par une preuve visuelle.

```
from PIL import Image
from vlm_engineering.clip import CLIPEncoder

image = Image.open("data/example.jpg").convert("RGB")
clip = CLIPEncoder()
print(clip.zero_shot_classify(image, ["chat", "chien", "avion"]))
```

## 3. Qwen3-VL-2B-Instruct

Qwen/Qwen3-VL-2B-Instruct est un modele image-text-to-text publie sous licence Apache-2.0. La page Hugging Face actuelle expose un pipeline image-text-to-text ainsi qu'un chargement direct avec AutoProcessor et AutoModelForMultimodalLM.

| Cas d'usage               | Contribution du VLM                                                |
|---------------------------|--------------------------------------------------------------------|
| Description d'image       | Caption riche et factuelle plutot qu'un simple score de similarite |
| Question-reponse visuelle | Repondre a une question en se fondant sur une ou plusieurs images  |
| Analyse de diagramme      | Recuperer composants, fleches, groupes et relations spatiales      |
| Comprehension de capture  | Interpréter dashboards, etats UI et formulaires                    |
| Tableaux/documents        | Combiner texte visible et structure de mise en page                |
| Extraction structuree     | Retourner du JSON valable par le code                              |

## 4. Trois modes de chargement

### 4.1 Mode Hub/cache

```
from vlm_engineering.qwen import QwenVLModel

model = QwenVLModel.from_hub("Qwen/Qwen3-VL-2B-Instruct")
print(model.generate("data/example.jpg", "Decris cette image avec precision."))
```

Le premier lancement peut telecharger les poids. Les executions suivantes reutilisent le cache Hugging Face.

### 4.2 Mode local/hors-ligne explicite

```
vlm-lab download-model \
  --model-id Qwen/Qwen3-VL-2B-Instruct \
  --output models/Qwen3-VL-2B-Instruct

vlm-lab describe data/example.jpg \
  --model-path models/Qwen3-VL-2B-Instruct
```

Le chargeur local fixe `local_files_only=True`. Ce mode est utile pour des deploiements reproductibles, des environnements isolés et une gestion explicite du cycle de vie du modèle.

### 4.3 `trust_remote_code`

Le projet garde `trust_remote_code=False` par défaut. Qwen3-VL est actuellement intégré nativement dans Transformers, donc l'usage normal n'exige pas l'exécution de code Python fourni par le dépôt du modèle. Activez cette option uniquement pour un modèle de confiance qui en a réellement besoin.

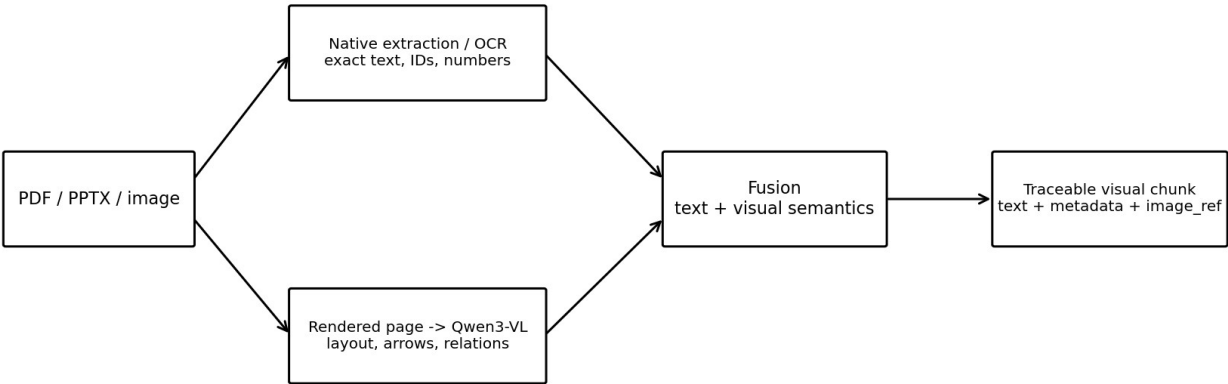
## 5. Comprehension visuelle structuree pour le RAG

Pour un RAG, demander seulement 'decris l'image' est souvent trop vague. Il vaut mieux demander des champs stables, validables et indexables.

```
{
  "page_type": "diagram|table|text|mixed|photo|screenshot",
  "title": "...",
  "summary": "...",
  "entities": [...],
  "relations": [{"source": "...", "relation": "...", "target": "...", "evidence": "...}],
  "important_text": [...],
  "uncertainties": [...]}
}
```

Les incertitudes doivent être explicites : une extraction utile dit 'flèche illisible' plutôt que d'inventer une relation avec confiance.

# 6. Le VLM doit-il remplacer l'OCR ?



En general, il ne faut pas penser l'architecture comme VLM contre OCR. L'extraction native/OCR est souvent meilleure pour identifiants exacts, nombres et longs textes. Le VLM est meilleur pour relations visuelles, mise en page, groupes, diagrammes et pages mixtes. On fusionne les deux en conservant la provenance.

- Utiliser d'abord le parsing natif/OCR quand du texte littéral fiable est disponible.
- Rendre chaque page/slide en image pour que le VLM voie la structure.
- Au debut appeler le VLM partout, puis ajouter un selecteur lors du passage a l'echelle.
- Conserver separement texte natif et sortie VLM avant la fusion.
- Garder image\_ref afin que le VLM final puisse revoir la preuve visuelle d'origine.

# 7. Chunks visuels semantiques

Un bon chunk est une unite autonome de sens, pas seulement une fenetre fixe de tokens. Un diagramme doit rester lie a son titre, sa legende, ses entites, ses relations, son contexte parent et son image d'origine.

| Champ                | Role                                            |
|----------------------|-------------------------------------------------|
| chunk_id             | Tracabilite stable                              |
| document_id / page   | Retour a la source                              |
| chunk_type           | diagram, table, mixed, screenshot...            |
| text                 | Texte natif + visuel fusionne pour le retrieval |
| entities / relations | Semantique structuree                           |
| image_ref            | Preuve visuelle originale                       |
| parent_context       | Hierarchie lors du sous-chunking                |

Pour le texte narratif, un overlap limite peut aider. Pour un diagramme structure, il vaut mieux un contexte parent-enfant que couper arbitrairement une relation en deux.

# 8. RAG visuel avec seulement un embedder texte

Un VLM peut servir de pont entre un corpus visuel et une pile RAG existante uniquement textuelle :

image/page -> description + JSON Qwen3-VL -> texte de retrieval -> embedder texte -> vector DB

Cette approche est pratique quand l'organisation possede deja un bon embedder texte. Le retrieval ne pourra cependant retrouver que les informations visuelles que la description VLM a effectivement exprimees. Conservez l'image originale pour le grounding final.

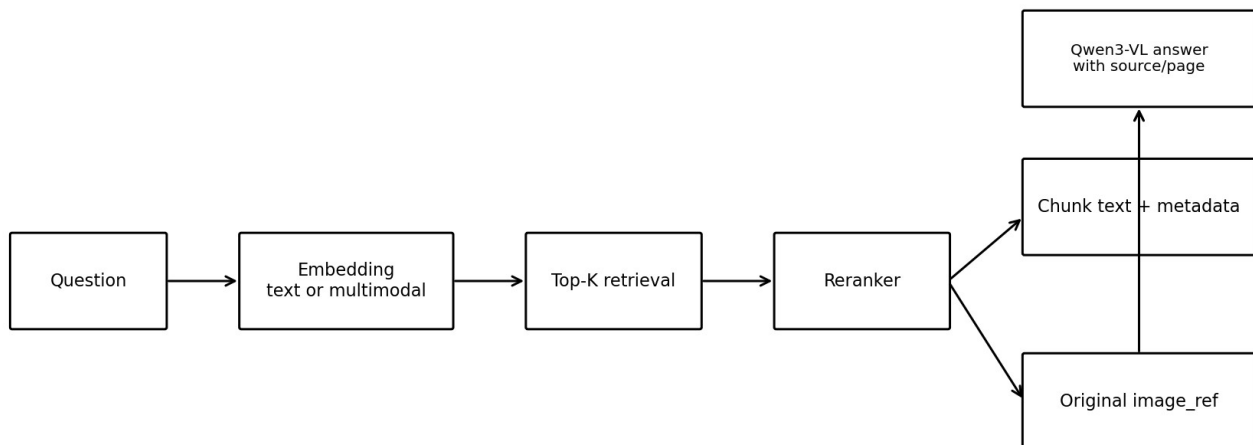
## 9. Retrieval reellement multimodal

Qwen3-VL-Embedding-2B est concu pour le retrieval multimodal et peut représenter texte, images, captures, video ou melanges texte+image. Son integration SentenceTransformers constitue le chemin principal dans ce projet. La fiche modele indique jusqu'a 2048 dimensions et un contexte de 32k.

```
from vlm_engineering.retrieval import QwenMultimodalEmbedder

embedder = QwenMultimodalEmbedder()
queries = ["Trouve le schema contenant un cache et une base de donnees."]
docs = ["note texte", "data/architecture.png", {"text": "Architecture paiement",
"image": "data/architecture.png"}]
q = embedder.encode(queries, prompt="Retrieve relevant technical document evidence.")
d = embedder.encode(docs)
print(q @ d.T)
```

## 10. Reranking et reponse finale fondee sur les preuves



Qwen3-VL-Reranker-2B est l'etage de precision : l'embedder effectue un rappel large puis le reranker note plus finement les paires requete-document. On recupere ensuite texte du chunk et image\_ref avant de donner les deux a Qwen3-VL-Instruct pour la reponse finale.

```
from vlm_engineering.retrieval import QwenMultimodalReranker

reranker = QwenMultimodalReranker()
scores = reranker.score("Quel schema montre PostgreSQL ?", candidates)
```

## 11. Evaluation pour la production

| Couche              | Metriques utiles                                                                  |
|---------------------|-----------------------------------------------------------------------------------|
| Extraction visuelle | taux JSON valide, exact match/F1 entites-relations, taux de relations hallucinees |
| Retrieval           | Recall@K, MRR, taux de bonne image/page                                           |
| Reranking           | MRR / nDCG / pertinence top-1                                                     |
| Reponse finale      | exactitude fondee sur preuves, citation source/page, abstention en cas ambigu     |
| Operations          | latence, memoire GPU, debit, taux echec, version modele/prompt                    |

Commencez avec 30 a 100 pages et questions representatives. Un bon jeu d'evaluation et un schema de donnees tracable valent souvent plus qu'un changement immediat vers un modele plus gros.

## 12. Demarrage rapide du projet

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -e ".[all,dev,notebooks]"
```

```
python -m ruff check src tests examples
python -m mypy src/vlm_engineering
python -m pytest
```

```
vlm-lab describe data/example.jpg
vlm-lab analyze data/page_001.png
```

## 13. Exercices pratiques

1. Executer une classification CLIP zero-shot sur une image et observer la sensibilite aux labels.
2. Poser la meme question a Qwen3-VL avec un prompt vague puis un prompt structure ; comparer la reutilisabilite.
3. Sur un diagramme technique, comparer texte OCR/natif avec extraction VLM des entites et relations.
4. Creer des chunks visuels pour dix pages et mesurer si le retrieval texte retrouve mieux les faits visuels apres ajout des descriptions VLM.
5. Remplacer l'embedder texte par Qwen3-VL-Embedding et comparer Recall@K.
6. Ajouter le reranking et mesurer si la bonne page remonte dans le classement.

## 14. Sources et fraicheur

- Qwen/Qwen3-VL-2B-Instruct - <https://huggingface.co/Qwen/Qwen3-VL-2B-Instruct>
- Qwen/Qwen3-VL-Embedding-2B - <https://huggingface.co/Qwen/Qwen3-VL-Embedding-2B>
- Qwen/Qwen3-VL-Reranker-2B - <https://huggingface.co/Qwen/Qwen3-VL-Reranker-2B>
- Guide Hugging Face Hub - [https://huggingface.co/docs/huggingface\\_hub/guides/download](https://huggingface.co/docs/huggingface_hub/guides/download)
- Base pedagogique fournie avec ce projet : cours VLM, chunking semantique et RAG multimodal (PDF francais de 28 pages).

Les interfaces et recommandations evoluent. En production, epinglez les revisions des modeles, versionnez les prompts et relancez l'evaluation avant chaque mise a jour.